

Exp No: 24 MODELING CLOCK-DRIVEN CONCURRENT PROCESSES USING SC_THREAD AND WAIT() IN SYSTEMC

AIM: To understand and implement a clock-driven process using: • sc_clock • SC_THREAD • wait() • Event sensitivity (positive edge trigger) in SystemC.

APPARATUS/SOFTWARE REQUIRED:

- Linux OS / WSL & Ubuntu
- systemC

PROGRAM: #include SC_MODULE(ClockExample)

```
{
  sc_in clk; void process()
{
  while (true)
  {
    wait();
    std::cout << "Time: " << sc_time_stamp()
    << " - Clock Triggered" << std::endl;
  }
}
SC_CTOR(ClockExample) {
  SC_THREAD(process);
  sensitive << clk.pos();
}
};
int sc_main(int argc, char* argv[])
{
  sc_clock clock("clock", 1, SC_NS);
  ClockExample module("module");
  module.clk(clock);
  sc_start(5, SC_NS);
  return 0;
}
```

OUTPUT:

The screenshot displays the EDA Playground interface. On the left, a sidebar contains navigation links for 'Languages & Libraries' (Testbench + Design, Libraries, Tools & Simulators) and 'Examples'. The main workspace is divided into two code editors: 'testbench.cpp' and 'design.cpp'. The 'testbench.cpp' editor contains the following C++ code:

```
1 #include <systemc>
2 using namespace sc_core;
3 using namespace std;
4
5 SC_MODULE(ClockExample) {
6     sc_in<bool> clk;
7
8     void process() {
9         while (true) {
10             wait();
11             cout << "Time: " << sc_time_stamp()
12                 << " - Clock Triggered" << endl;
13         }
14     }
15
16     SC_CTOR(ClockExample) {
17         SC_THREAD(process);
18         sensitive << clk.pos();
19     }
20 };
21
22 int sc_main(int argc, char* argv[]) {
23     sc_clock clock("clock", 1, SC_NS);
24
25     ClockExample module("module");
26     module.clk(clock);
27
28     sc_start(5, SC_NS);
29     return 0;
30 }
```

The 'design.cpp' editor is currently empty, showing only placeholder comments. Below the code editors, a 'Log' tab is active, displaying the compilation and execution output:

```
[2020-08-23 08:26:49 UTC] g++ -std=c++14 -lsystemc *.cpp && echo "Compile done. Starting run..." && ./sim
Compile done. Starting run...

SystemC 2.3.3-Accellera --- May 23 2020 14:33:56
Copyright (c) 1996-2018 by all Contributors,
ALL RIGHTS RESERVED

Time: 0 s - Clock Triggered
Time: 1 ns - Clock Triggered
Time: 2 ns - Clock Triggered
Time: 3 ns - Clock Triggered
Time: 4 ns - Clock Triggered
done
```

An 'Activate Windows' watermark is visible in the bottom right corner of the output window.

RESULT:

Thus, the clock-driven process using SC_THREAD and wait() was successfully implemented and verified in SystemC